

Code Review: AFSK Transceiver + Flutter Client

Отчёт по выявленным проблемам и багам

Дата: 23 августа 2026

Сводная таблица приоритетов

Приоритет	Модуль	Проблема	Краткое исправление
P0 — Критично	afsk_decoder.c	PLL wrap-around при коррекции фазы: приведение (int32_t) к uint32_t сбрасывает фазу в 0 при значениях > 0x7FFFFFFF	Убрать приведение (int32_t), использовать (float)dec->pll
P0 — Критично	afsk_decoder.c	Race condition при инициализации sine table: двойная инициализация при одновременном доступе с двух ядер	Заменить на atomic_bool или защитить мьютексом
P1 — Высокий	tx_ad9851.c	tx_ad9851_wait_idle может повиснуть навсегда, если ISR успевает отработать до вызова wait_idle	Проверять tx_is_active перед xSemaphoreTake, иначе не ждать
P1 — Высокий	afsk_decoder.c	have_prev_sign не сбрасывается при abort_frame: ложный детект перехода на первом бите нового фрейма	Сбрасывать prev_sign и have_prev_sign в abort_frame()
P2 — Средний	wifi_link.c	Отсутствие rate limiting на WebSocket: клиент может забить s_tx_queue (длина 4)	Добавить троттлинг: не более 1 сообщения в 100 мс с одного fd
P2 — Средний	Flutter (screen_pro)	Бесконечный рост списка _messages: память не освобождается при длительной работе	Ограничить список 500 сообщениями, старые удалять

P2 — Средний	main.c	console_task и rx_task на одном ядре (core 1): консольный ввод может вытеснять RX, теряя I2S-данные	Перенести console_task на core 0
P3 — Низкий	afsk_protocol.c	Скрытая зависимость от глобальной tx_bit_r_idx	Передать r_idx как параметр функции

1. AFSK Decoder (afsk_decoder.c / .h)

1.1 P0 — PLL wrap-around при коррекции фазы

При значении фазы $> 0x7FFFFFFF$ приведение к `int32_t` даёт отрицательное число. Умножение на `PLL_INERTIA` и обратное приведение к `uint32_t` сбрасывает фазу в ноль. Битовый такт «прыгает», декодер теряет синхронизацию.

Проблемный код:

```
dec->pll = (uint32_t)((float)(int32_t)dec->pll * PLL_INERTIA);
```

Исправление:

```
dec->pll = (uint32_t)((float)dec->pll * PLL_INERTIA);
```

1.2 P0 — Race condition при инициализации sine table

Функция `afsk_decoder_tables_init()` проверяет флаг `sine_table_ready` без атомарных операций. При одновременной инициализации двух декодеров на разных ядрах таблица может строиться дважды одновременно, что приводит к повреждению данных.

Исправление: использовать `atomic_bool` (C11) или защитить мьютексом.

1.3 P1 — have_prev_sign не сбрасывается при abort_frame

При сбросе фрейма (`abort_frame`) не сбрасываются поля `prev_sign` и `have_prev_sign`. На первом бите нового фрейма может быть ложно детектирован «переход», и PLL дернёт фазу в неверную сторону.

Исправление: добавить в `abort_frame()`:

```
dec->prev_sign = false; dec->have_prev_sign = false;
```

1.4 P2 — Переполнение rx_buffer

Если принято > 255 байт полезной нагрузки, `rx_index` застревает на `MAX_MESSAGE_LEN - 1`, но биты продолжают поступать. CRC вычисляется по первым 255 байтам, следующий байт становится «CRC». Декодер не сообщает об ошибке, а выдаёт обрезанные данные.

Исправление: вызывать `abort_frame("payload overflow")` при `rx_index >= MAX_MESSAGE_LEN - 1`.

2. TX AD9851 (tx_ad9851.c / .h)

2.1 P1 — tx_ad9851_wait_idle может повиснуть навсегда

Если между tx_ad9851_send_block (возвращает true) и tx_ad9851_wait_idle таймер успевает отработать, tx_is_active станет false, семафор не будет выдан, и задача зависнет на portMAX_DELAY.

Проблемный код:

```
if (tx_is_active && tx_idle_sem) { xSemaphoreTake(tx_idle_sem, portMAX_DELAY); }
```

Исправление: добавить else-ветку или проверять tx_is_active внутри цикла с таймаутом.

2.2 P3 — Неоптимальная очистка буфера

Цикл по 4096 элементам с присвоением false вместо memset. На ESP32 это не критично, но лишний код.

```
Исправление: memset((void*)tx_bit_buffer, 0, sizeof(tx_bit_buffer));
```

3. AFSK Protocol (afsk_protocol.c / .h)

3.1 P3 — Скрытая зависимость от глобальной tx_bit_r_idx

Функция `afsk_serialize_block` зависит от `extern volatile uint32_t tx_bit_r_idx`. Это скрытая связь, усложняющая тестирование и переиспользование.

Исправление: передавать `r_idx` как параметр:

```
void afsk_serialize_block(..., volatile uint32_t *r_idx);
```

3.2 P2 — Edge case в `afsk_utf8_block_len`

Если `start` указывает на continuation byte (нарушение инварианта), функция вернёт `max_len` и разрежет символ. Нет защиты от некорректного входа.

Исправление: добавить `assert` или проверку валидности `start`.

4. Wi-Fi Link (wifi_link.c / .h)

4.1 P2 — Отсутствие rate limiting на WebSocket

Любой клиент может завалить s_tx_queue (длина 4) сообщениями. При переполнении очереди сообщения отбрасываются без уведомления отправителя (кроме System-нотификации).

Исправление: добавить троттлинг на fd — не более 1 сообщения в 100 мс с одного клиента.

4.2 P2 — Отсутствие CORS

Если потребуется веб-клиент в браузере, POST-запросы с другого origin будут заблокированы. Для текущего мобильного приложения не критично, но ограничивает расширяемость.

4.3 P2 — find_param не robust

Использование strstr на теле запроса без учёта длины. При бинарных данных (хотя формат form-urlencoded) возможен выход за границы буфера.

Исправление: использовать strnstr или явно передавать длину тела.

4.4 P3 — ws_close_fn вызывает close(sockfd)

В ESP-IDF close_fn заменяет стандартное закрытие сокета. Убедитесь, что httpd не пытается закрыть тот же fd повторно (в старых версиях IDF возможен double-close).

5. main.c

5.1 P2 — console_task и rx_task на одном ядре

Обе задачи закреплены на core 1. Активный консольный ввод может вытеснять rx_task, что приводит к переполнению DMA-буфера I2S и потере аудиосэмплов.

Исправление: перенести console_task на core 0 или повысить приоритет rx_task выше console_task.

5.2 P2 — Хрупкая экстракция I2S-сэмплов

Сдвиг >> 8 зависит от bit_shift = true в конфигурации I2S. При изменении конфигурации (например, bit_shift = false) декодер сломается без явной ошибки компиляции.

Исправление: добавить static_assert или комментарий, привязанный к конфигу I2S.

5.3 P3 — Рассогласование таймаутов

SIGNAL_TIMEOUT_MS = 2000 в декодере, MESSAGE_IDLE_MS ≈ 8 сек для блока 50 байт. Если BLOCK_GAP_MS когда-нибудь увеличат > 1500 мс, декодер начнёт резать сообщения на части.

Исправление: добавить assert(BLOCK_GAP_MS < SIGNAL_TIMEOUT_MS) или сделать MESSAGE_IDLE_MS динамическим.

6. Flutter-клиент (main.txt / screen_pro.txt)

6.1 P2 — Бесконечный рост _messages

Список сообщений никогда не очищается. При длительной работе (чат-лог) память растёт линейно, что приведёт к OOM на слабых устройствах.

Исправление: ограничить список 500 сообщениями, старые удалять при превышении лимита.

6.2 P2 — Агрессивный poll подключения

Каждые 5 секунд вызывается getWifiP() + HTTP ping + возможное WS-переподключение. На Android getWifiP() может будить радио, увеличивая энергопотребление.

Исправление: увеличить интервал до 10 сек при stable error, добавить экспоненциальный backoff.

6.3 P2 — Нет валидации from в POST

send_post_handler принимает from из тела запроса без санитизации. Злоумышленник может отправить from=System: и подменить системные уведомления.

Исправление: запретить ':' и префикс 'System' в имени на стороне прошивки (для POST-обработчика).

6.4 P2 — _PendingEcho: точное совпадение строк

Сопоставление по точной строке frame == message. Если на сервере добавится/уберётся trim(), синхронизация эхо сломается.

Исправление: добавить sequence number или UUID к сообщению на стороне клиента и сервера.

6.5 P3 — _echoTimeout = 6 секунд

Для 300 бод передача 300 символов занимает ~10 сек + preamble 3.2 сек. Хотя эхо приходит после постановки в очередь, при забитой очереди s_tx_queue сообщение может задержаться.

Исправление: сделать таймаут адаптивным: базовый 5 сек + 1 сек на каждые 50 символов.

6.6 P3 — Монолитный ChatScreen

Весь state, сеть, UI и логика в одном классе (~650 строк). При расширении (несколько чатов, offline-очередь) поддержка станет сложной.

Исправление: выделить ChatRepository, MessageBloc/ChangeNotifier, ConnectionCubit.

6.7 P3 — animateTo(0) на длинном списке

_scrollToBottom использует animateTo(0) с длительностью 200 мс. На очень длинном списке анимация может лагать.

Исправление: использовать `jumpTo(0)` для мгновенной прокрутки или ограничить длину списка.